Robot Ping-Pong Control Under Physics Variation: Domain Randomization and Residual Reinforcement Learning

Julia Novick* Jonathan Zhang* Mayand Gulati* Sagnik Biswas*

UC Santa Barbara

March 18, 2026

Abstract

We study residual reinforcement learning for robotic ball bouncing under simulation physics uncertainty. The controller combines a finite-state-machine and inverse-kinematics baseline (FSM+IK) with a learned Soft Actor-Critic (SAC) residual that provides bounded joint-space corrections. We compare nominal SAC training on fixed physics against robust SAC training with per-episode domain randomization over mass, friction, and restitution parameters. Experiments use 20-episode deterministic evaluations at checkpoints (500k, 750k, 1M training steps) under nominal and randomized test physics in the Drake simulator. Cross-simulator MuJoCo evaluations are nominal-only (500 episodes). Nominal SAC achieves the strongest absolute performance in Drake (482 mean hits under nominal physics; 328 under randomized), while robust SAC exhibits better relative robustness trends at later checkpoints (+25.9% at 750k and +48.5% at 1M from nominal to randomized evaluation). In nominal MuJoCo sim-to-sim transfer, all policies degrade relative to Drake; the FSM baseline slightly outperforms robust SAC, while robust SAC remains close and outperforms nominal SAC. We further analyze how the residual scale parameter governs the degree of RL correction injected on top of the baseline, finding that a narrow optimal range ($\alpha \approx 0.01$) yields peak hit counts while larger or smaller values degrade performance. Code and artifacts are public at https://github.com/MG-05/ping-pong-domain-randomization.

1 Introduction

Robot ball bouncing is hard because frequent contacts amplify small model errors. In this project, we evaluate policies not only by nominal return, but also by how well they handle physically plausible perturbations.

Our system uses a hybrid architecture:

- an analytic inner loop (FSM+IK) for structured intercept and strike behavior,
- a learned residual outer loop (SAC) for corrective adaptation.

This setup lets us ask a direct question: when does residual learning help, and how does domain randomization change generalization?

We evaluate three research questions:

- **RQ1 (Absolute Performance):** Can residual SAC exceed the hand-engineered baseline?
- **RQ2 (Robustness):** Does domain-randomized training improve behavior under perturbed physics?
- **RQ3 (Stability):** What do variance and survival metrics reveal about policy reliability?

2 Related Work

Entropy-regularized RL. SAC optimizes return plus policy entropy, giving strong continuous-control performance and improved training stability [1].

Continuous-control actor-critic lineage. Deterministic actor-critic methods such as DDPG [2] show the utility of off-policy replay-based learning in robotics, but can be brittle under noise and hyperparameter shifts.

Domain randomization. Randomizing simulator dynamics can reduce overfitting to a single simulator instance and improve out-of-distribution robustness [3, 4].

Robust and residual RL. Robust RL emphasizes performance under perturbations [5]; residual RL combines structured control priors with learned corrections for safety and sample efficiency [6]. Recent examples include SAC-GMM for trajectory-space skill refinement [7], residual adaptation of DMPs for contact-rich manipulation [8], and context-encoded residual adaptation for offline-to-online RL under changing dynamics [9].

Practical implementation framework. Stable-Baselines3 provides reproducible, standard RL training infrastructure that we leverage for SAC training and evaluation orchestration [10].

*All authors contributed equally to this work.

3 Theoretical Background

3.1 MDP and Residual Action Parameterization

Let $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$ denote the Markov decision process [11], where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s'|s,a)$ is the transition kernel, $r(s,a)$ is the reward function, and $\gamma \in [0,1)$ is the discount factor. Here, $s_t \in \mathbb{R}^{20}$ contains the robot joint positions and velocities (14 dimensions) plus the ball Cartesian position and velocity (6 dimensions), and $a_t \in [-1,1]^7$ is a normalized residual action where each component corresponds to one of the seven KUKA iiwa joint axes.

Baseline and residual are combined to produce the commanded joint position vector as follows:

$$q_t^{\text{cmd}} = \Pi_{\mathcal{Q}}\big(q_t^{\text{base}} + \alpha\,\delta\,\text{clip}(a_t, -1, 1)\big)\,,$$

where:

- $q_t^{\text{cmd}} \in \mathbb{R}^7$ is the final joint-position command sent to the robot at time step t,
- $q_t^{\text{base}} \in \mathbb{R}^7$ is the baseline command produced by the FSM+IK controller (Section 6.7),
- $\alpha \in [0,1]$ is the *residual scale*, controlling how much of the learned correction is injected (Section 7),
- $\delta = 0.15$ rad is the maximum residual magnitude per joint,
- $\text{clip}(a_t, -1, 1)$ clamps each component of the policy output to $[-1,1]$,
- $\Pi_{\mathcal{Q}}$ projects the result onto the feasible joint-limit set $\mathcal{Q} = \{q : q^{\min} \le q \le q^{\max}\}$.

3.2 SAC Objective

SAC solves an entropy-regularized objective

$$J(\pi) = \mathbb{E}_{\tau\sim\pi}\left[\sum_{t=0}^{\infty} \gamma^t \left(r_t + \beta\mathcal{H}(\pi(\cdot|s_t))\right)\right],$$

where $\tau = (s_0, a_0, r_0, s_1, \ldots)$ is a trajectory sampled under policy π, $r_t = r(s_t, a_t)$ is the per-step reward, $\beta > 0$ is the entropy temperature (auto-tuned in our implementation), and $\mathcal{H}(\pi(\cdot|s_t)) = -\mathbb{E}_{a\sim\pi}[\log \pi(a|s_t)]$ is the policy entropy at state s_t.

The soft Bellman operator defines the soft Q-function Q^π and soft value function V^π:

$$Q^\pi(s_t, a_t) = r_t + \gamma\,\mathbb{E}_{s_{t+1}\sim P}[V^\pi(s_{t+1})]\,,$$
$$V^\pi(s) = \mathbb{E}_{a\sim\pi}[Q^\pi(s,a) - \beta \log \pi(a|s)]\,.$$

3.3 Why Residual SAC Is Appropriate for This Task

Robotic ball bouncing involves known geometric structure (intercept timing and IK). Learning full control from scratch is unnecessary and increases exploration burden. Residual SAC addresses a narrower problem: improving a feasible baseline policy via bounded corrections.

The feasible joint-limit set $\mathcal{Q} = \{q \in \mathbb{R}^7 : q_i^{\min} \le q_i \le q_i^{\max},\ i = 1, \ldots, 7\}$ is the Cartesian product of per-joint intervals defined by the KUKA iiwa hardware limits, further tightened by a configurable deviation bound around the home configuration.

Remark 1. *Because residual exploration is bounded by $\alpha\delta$ per joint and then projected onto $\mathcal{Q}$, the residual policy cannot directly violate joint-position constraints. A full proof is given in Appendix A.*

3.4 Robustness Under Transition Shift

Let P_{nom} and P_{rand} denote the nominal and randomized transition kernels respectively, where $P_{\text{nom}}(s'|s,a)$ governs state transitions under fixed (default) physics parameters, and $P_{\text{rand}}(s'|s,a)$ governs transitions under per-episode randomized physics. Training only on P_{nom} may maximize nominal return while suffering *performance degradation*—a drop in mean hit count and survival rate—under P_{rand}. Domain-randomized training approximates risk minimization over the distribution of physics perturbations, specifically minimizing the expected drop in cumulative reward across the randomization envelope.

3.5 SAC Training Dynamics in Residual Control

For this project, SAC optimization is best interpreted as learning a correction distribution over a moving baseline controller output. Unlike standard end-to-end policy learning, the action semantics are locally corrective: each action dimension perturbs one joint around a feasible IK-informed command. This affects critic learning in two ways.

First, replay buffer support is naturally concentrated around physically plausible commands, because the baseline controller prevents the robot from reaching states far from the kinematic manifold of valid arm configurations (i.e., joint configurations reachable through the FSM's IK-planned trajectories). Second, value approximation error can still accumulate due to contact discontinuities (ball-paddle impacts), leading to the non-monotonic checkpoint behavior observed in our results.

In actor-critic form, SAC alternates:

- **critic updates**: regress soft Bellman targets under replayed transitions;
- **actor updates**: maximize soft value while preserving entropy;
- **target updates**: Polyak averaging controlled by $\tau = 0.005$.

Because the reward combines sparse event terms (new hit bonus of +5.0) and dense shaping terms (alive bonus of +0.01 per step, apex-tracking bonus), critic targets mix

abrupt and smooth components. This mixed structure plausibly contributes to temporary mismatch between the critic's value estimates and the true returns at intermediate checkpoints (e.g., the 750k dip in Figure 4a).

3.6 Entropy Temperature and Exploration Interpretation

In SAC, entropy regularization controls exploration intensity. In a residual-control setting, higher entropy does not mean arbitrary behavior in task space; it means diverse local perturbations around structured baseline commands. This can be advantageous when baseline timing is approximately correct but contact quality requires adaptation.

However, entropy-driven exploration can still produce transiently suboptimal residual patterns before converging to better correction strategies. The 500k $\rightarrow$ 750k dip followed by 1M rebound in both nominal and robust tracks (Figure 4a) is consistent with such reorganization.

3.7 Critic and Actor Losses

SAC alternates stochastic gradient steps on the twin critic loss and the actor loss. The critic loss for parameters ϕ is:

$$\mathcal{L}_Q(\phi) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(Q_\phi(s,a) - y(r,s')\right)^2\right],$$
$$y(r,s') = r + \gamma\left(\min_{i\in\{1,2\}} Q_{\bar{\phi}_i}(s',a') - \beta\log\pi_\theta(a'|s')\right),$$

where $\mathcal{D}$ is the replay buffer of past transitions (s,a,r,s'), $Q_{\bar{\phi}_i}$ are the two target critic networks with parameters $\bar{\phi}_i$ updated via Polyak averaging, $a' \sim \pi_\theta(\cdot|s')$ is an action sampled from the current policy at the next state, and β is the entropy temperature. The actor objective for parameters θ is:

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_{s\sim\mathcal{D},\, a\sim\pi_\theta}\left[\beta\log\pi_\theta(a|s) - \min_i Q_{\phi_i}(s,a)\right].$$

3.8 Distributional Robustness Perspective

Let $\xi = (\text{ball mass}, \text{friction}, \text{restitution}, \ldots)$ denote the vector of randomized physics parameters sampled from a distribution $\mathcal{P}_\Xi$ defined by the ranges in Table 1. Nominal training optimizes expected return under a single fixed parameter vector ξ_0:

$$\max_\pi \; \mathbb{E}_{\tau\sim P_{\xi_0},\pi}[R(\tau)],$$

where $R(\tau) = \sum_{t=0}^{T}\gamma^t r_t$ is the discounted cumulative reward along trajectory τ, and P_{ξ_0} is the transition kernel induced by the nominal physics ξ_0.

Robust (domain-randomized) training instead optimizes the expected return averaged over the physics distribution:

$$\max_\pi \; \mathbb{E}_{\xi\sim\mathcal{P}_\Xi}\mathbb{E}_{\tau\sim P_\xi,\pi}[R(\tau)].$$

This does not guarantee worst-case optimality, but it better aligns optimization with deployment where physics are uncertain.

Proposition 1 (Expected-risk interpretation)**.** *If test-time dynamics are sampled from $\mathcal{P}_\Xi$, then robust-domain-randomized training is an unbiased Monte Carlo estimator of expected return under that distribution (up to function-approximation error and finite sample effects).*

Practical implication: robust SAC can improve average behavior across perturbations even when it does not maximize peak nominal performance. A full proof is provided in Appendix A.

4 Methods and Implementation

4.1 Environment and Timing

The environment (`src/envs/residual_env.py`) runs physics at 1 ms time steps (the Drake simulator's internal integration step) and applies RL control actions at 10 ms intervals. This 10:1 time-scale separation preserves contact fidelity in the physics engine while keeping policy inference tractable—the RL agent only needs to produce an action every 10 ms rather than every physics step.

4.2 Observation, Action, and Interfaces

Observation (20-D): The observation vector at each control step is

$$o_t = \Big[\underbrace{q_{1:7}}_{\text{joint pos.}},\ \underbrace{\dot{q}_{1:7}}_{\text{joint vel.}},\ \underbrace{p_b^x, p_b^y, p_b^z}_{\text{ball pos.}},\ \underbrace{v_b^x, v_b^y, v_b^z}_{\text{ball vel.}}\Big],$$

where $q_{1:7} \in \mathbb{R}^7$ are the KUKA iiwa joint angles (rad), $\dot{q}_{1:7} \in \mathbb{R}^7$ are the corresponding joint velocities (rad/s), $p_b \in \mathbb{R}^3$ is the ball center-of-mass position in the world frame (m), and $v_b \in \mathbb{R}^3$ is the ball linear velocity (m/s).

Action (7-D): The policy outputs a normalized residual $a_t \in [-1,1]^7$, where each component $a_t^{(i)}$ maps to a bounded joint offset $\alpha\cdot\delta\cdot a_t^{(i)}$ on joint i of the iiwa arm (with $\alpha = 0.5$ and $\delta = 0.15$ rad by default).

At each control step (every 10 ms):

1. Evaluate the FSM+IK controller (Section 6.7) to get the baseline joint command $q_t^{\text{base}} \in \mathbb{R}^7$.
2. Query the residual SAC policy for normalized action $a_t \in [-1,1]^7$ and compute the corrected command $q_t^{\text{cmd}} = \Pi_{\mathcal{Q}}(q_t^{\text{base}} + \alpha\delta\,\text{clip}(a_t))$.
3. Send q_t^{cmd} as a position command to the iiwa joint position controller (a Drake PD controller that tracks the desired joint positions by generating appropriate joint torques).
4. Advance the Drake simulator by one control interval (10 ms, i.e., 10 physics steps at 1 ms each) and compute the reward and termination signals.

4.3 Reward Construction

The reward function at each control step combines four terms, designed to balance persistence, contact productivity, and bounce quality:

- **Alive bonus** ($r_{\text{alive}} = 0.01$): awarded at every step where the ball height p_b^z remains above the minimum threshold $z_{\min} = 0.08\,\text{m}$.
- **Hit bonus** ($r_{\text{hit}} = 5.0$): awarded for each new debounced paddle-ball contact event, with a cooldown of 10 control steps (100 ms) between consecutive hit detections to prevent double-counting a single contact.
- **Apex-tracking bonus**: $r_{\text{apex}} \cdot \exp(-10 \cdot e_{\text{apex}}^2)$, where $r_{\text{apex}} = 2.0$ is the maximum apex reward and $e_{\text{apex}} = |p_b^z - z_{\text{target}}|$ is the absolute error between the ball's height at its trajectory apex and the target apex height $z_{\text{target}} = 0.55\,\text{m}$. This bonus is triggered when the ball's vertical velocity crosses zero from positive to negative (i.e., the ball reaches its peak).
- **Drop penalty** ($r_{\text{drop}} = -5.0$): applied when the ball falls below $z_{\min}$, which also terminates the episode.

4.4 Training Configuration

From `src/train_rl.py`, SAC is configured with:

- MLP policy with two hidden layers of 256 units each, $[256, 256]$, a standard SAC size that was stable in our runs.
- learning rate 3×10^{-4}, the usual SAC default, kept because optimization remained stable.
- batch size 256 transitions per gradient step, a practical stability/compute trade-off for off-policy updates.
- replay buffer capacity 10^5 transitions, large enough for transition diversity without unnecessary memory cost.
- discount factor $\gamma = 0.99$, target network Polyak rate $\tau = 0.005$, standard SAC settings for long-horizon credit assignment and stable target tracking.
- residual scale $\alpha = 0.5$, maximum residual $\delta = 0.15$ rad, set to allow meaningful correction while keeping actions bounded.

We kept this hyperparameter set fixed across nominal and robust tracks so that differences come from domain randomization, not retuning.

Two training tracks are used:

- **Nominal SAC:** trained under fixed (default) physics parameters.
- **Robust SAC:** trained with per-episode domain randomization over the ranges in Table 1.

4.5 Domain Randomization Mechanism

`src/utils/randomization.py` samples parameters uniformly from Table 1 and writes randomized model/scenario files before each robust episode. The floor model is also regenerated each episode to vary contact behavior. These perturbations change actual simulator artifacts, not just internal coefficients.

Table 1: Domain randomization ranges used in robust training.

Parameter	Min	Max
Ball mass (kg)	0.0020	0.0035
Ball friction	0.15	0.30
Ball restitution	0.85	0.95
Paddle mass (kg)	0.08	0.15
Paddle friction	0.20	0.60
Paddle restitution	0.80	1.00

5 Experimental Protocol

5.1 Hypotheses

- **H1:** Residual SAC can exceed FSM baseline hit counts.
- **H2:** Domain-randomized SAC improves relative perturbation robustness.
- **H3:** Learning dynamics are non-monotonic across checkpoints.

5.2 Evaluation Procedure

`src/evaluate.py` runs deterministic policy inference for 20 episodes per model-condition pair. Evaluated checkpoints: 500k, 750k, 1M training steps. Evaluated conditions: nominal and randomized physics.

5.3 Metrics

Definition 1 (Mean hit performance)**.** *For episode hits h_e (the number of debounced paddle-ball contacts in episode e), $H = \frac{1}{N}\sum_{e=1}^{N} h_e$, where N is the number of evaluation episodes.*

Definition 2 (Relative perturbation effect)**.**

$$\Delta(\%) = 100 \cdot \frac{H_{\text{rand}} - H_{\text{nom}}}{H_{\text{nom}}},$$

where H_{rand} and H_{nom} are the mean hits under randomized and nominal physics respectively. Positive Δ indicates the policy performs better *under randomized physics than under nominal.*

Definition 3 (Relative variability)**.**

$$\text{CV}(\%) = 100 \cdot \frac{\sigma_{\text{rand}}}{H_{\text{rand}}},$$

where σ_{rand} is the standard deviation of hit counts under randomized physics. Lower CV indicates more consistent performance across physics perturbations.

Assumption 1 (Controlled comparisons)**.** *Within each evaluation block, horizon, deterministic inference mode, and episode count are fixed.*

6 Implementation Walkthrough

6.1 End-to-End Dataflow

At runtime, one control iteration follows this sequence:

1. read simulator outputs: the iiwa state vector $[q_{1:7}, \dot{q}_{1:7}] \in \mathbb{R}^{14}$ (7 joint positions and 7 joint velocities of the KUKA iiwa arm), the ball state (position p_b and velocity v_b), and the ball contact-force vector (6-D generalized forces used for hit detection);
2. evaluate the FSM controller (Section 6.7) to generate q_t^{base};
3. query residual SAC policy for normalized action $a_t \in [-1, 1]^7$;
4. map a_t to bounded joint corrections: $\Delta q = \alpha \cdot \delta \cdot \text{clip}(a_t)$ and add to q_t^{base};
5. clamp to joint bounds via $\Pi_{\mathcal{Q}}$ and send the resulting position command q_t^{cmd} to the iiwa joint-position controller;
6. advance simulation by one control interval (10 ms) and compute reward/termination.

This split keeps intercept/strike logic in the FSM and local correction in SAC, which made failures easier to diagnose than in an end-to-end policy.

6.2 Why the Observation Space Is Structured This Way

The 20-D observation contains joint position/velocity ($q_{1:7}$, $\dot{q}_{1:7}$) and ball position/velocity (p_b, v_b). Conceptually, this is the minimum state needed to:

- infer arm configuration and motion trend,
- infer ball flight state for near-term intercept correction,
- avoid overfitting to simulator-internal artifacts.

No image stream is used; this keeps the problem in a dynamics-control regime rather than perception-heavy policy learning.

6.3 Reward Design Trade-offs

The reward (Section 4.3) blends sparse success events and dense shaping:

- sparse hit bonuses ($r_{\text{hit}} = 5.0$) produce strong task signal but can destabilize critic learning when contacts are rare;
- alive ($r_{\text{alive}} = 0.01$) and apex shaping terms reduce reward sparsity but may create local optima (e.g., keeping short rallies alive rather than maximizing long-term hit count);
- drop penalties ($r_{\text{drop}} = -5.0$) discourage catastrophic failure but can overweight terminal events if improperly tuned.

In this project, the combined reward appears effective for discovering high-hit trajectories at 1M steps, but also exhibits checkpoint volatility, suggesting potential future reward-scaling work.

6.4 Contact Event Debouncing and Measurement Semantics

Hit counting uses force-thresholded contact detection: a hit is registered when the ball contact-force magnitude exceeds 0.05 N *and* the ball is above 0.20 m, with a cooldown of 10 control steps (100 ms) between detections. So "hits" are filtered events, not every physics-step contact, and they exclude many floor bounces or incidental collisions. Reported hit-rate differences therefore reflect both policy behavior and this event definition.

6.5 Training Infrastructure and Reproducibility

The training loop uses a standard set of RL engineering controls:

- checkpoint callback for temporal model snapshots,
- eval callback for periodic deterministic policy evaluation,
- monitor-wrapped environments for episodic metrics,
- fixed seeds to improve experiment repeatability.

These controls enable post-hoc analysis of learning dynamics such as the 750k performance dip.

6.6 Domain Randomization as Scenario Generation

Randomization is implemented by sampling parameters and writing randomized model/scenario files at environment reset, including a regenerated floor model for episode-to-episode contact variation. As above, these perturbations change simulator artifacts, not just internal coefficients.

6.7 FSM Controller Architecture

The baseline controller is a six-state finite state machine (Figure 1) implemented in `src/controllers/fsm_controller.py`. Each state maps to a distinct phase of the ball-bouncing rally cycle:

- **WAIT**: holds the home joint configuration $q_{\text{home}} = [0, 0.6, 0, -1.75, 0, 1.0, 0]$ rad; monitors ball state. Transitions to PLAN when all of: (i) the ball is descending ($v_b^z < -0.12$ m/s), (ii) the ball is above minimum height ($p_b^z > 0.26$ m), (iii) the plan cooldown (30 ms since last plan) has elapsed, and (iv) a valid ballistic intercept exists at the target hit height.
- **PLAN**: predicts a ballistic intercept at the target hit height ($z_{\text{hit}} = 0.38$ m) using projectile kinematics (see below). Computes the desired paddle pose and strike normal, then solves IK for three waypoints (prehit, hit, follow-through). Transitions to PREHIT on IK success, or to RECOVER if IK fails for any waypoint.

- **PREHIT**: interpolates the arm from its current configuration toward the prehit waypoint using linear joint-space interpolation. May replan at 40 ms intervals if time permits (replan cutoff: 40 ms before strike). Transitions to STRIKE when the current time reaches the scheduled strike start time $t_{\text{strike}} = t_{\text{hit}} - t_{\text{lead}}$, where $t_{\text{lead}} \geq 60$ ms is the strike lead time. Transitions to RECOVER if the ball passes the predicted hit time without striking.
- **STRIKE**: executes a timed linear interpolation through prehit→hit→follow waypoints. Detects contact via force threshold (> 0.05 N with ball above workspace floor) or ball velocity reversal (vertical velocity sign change from negative to positive, indicating an upward rebound). Transitions to FOLLOW_THROUGH on hit detection or after the strike window expires ($t_{\text{tail}} \geq 50$ ms after t_{hit}).
- **FOLLOW_THROUGH**: holds the follow-through pose for $\sim$100 ms, then transitions to WAIT.
- **RECOVER**: returns to home pose q_{home} when planning fails or a strike is missed; transitions to WAIT once joint position error is below 0.45 rad and joint velocity magnitude is below 0.25 rad/s relative to the home configuration.

Ballistic intercept prediction. The PLAN state uses classical projectile kinematics under gravity to predict where the ball will reach the target hit height. Given ball position p_b and velocity v_b at the current time, the vertical trajectory under gravity $g = 9.81$ m/s^2 is modeled as:

$$z(t) = p_b^z + v_b^z \cdot t - \tfrac{1}{2} g\, t^2.$$

The FSM solves $z(t) = z_{\text{hit}}$ via the quadratic formula to find the time-to-intercept τ_{hit}. Among real positive roots in the valid horizon (60 ms $\leq \tau_{\text{hit}} \leq$ 1.1 s), it selects the earliest root where the ball is still descending. The predicted 3D intercept position is then:

$$p_{\text{hit}} = p_b + v_b \cdot \tau_{\text{hit}} + \begin{bmatrix} 0 \\ 0 \\ -\frac{1}{2} g\, \tau_{\text{hit}}^2 \end{bmatrix},$$

and the predicted velocity at impact is $v_{\text{hit}} = v_b + [0, 0, -g\,\tau_{\text{hit}}]^\top$. The desired paddle normal is computed from v_{hit} with a tilt gain (0.22) that angles the paddle against the ball's horizontal velocity component (up to 18° maximum tilt) to maintain a centered bounce.

Workspace and timing constraints. The predicted intercept must lie within the reachable workspace: $x \in [0.40, 1.00]$ m, $y \in [-0.55, 0.55]$ m, $z \in [0.22, 0.90]$ m. The prehit waypoint is placed 60 mm before the contact point along the paddle normal, and the follow-through waypoint is 40 mm past the contact point, ensuring a smooth swing trajectory through the contact point.

This FSM gives residual learning three useful priors: feasible trajectories from IK targets, strike timing aligned with predicted intercepts, and intended paddle-ball contact geometry. In practice, SAC then learns local improvements to a competent controller rather than the whole task from scratch.

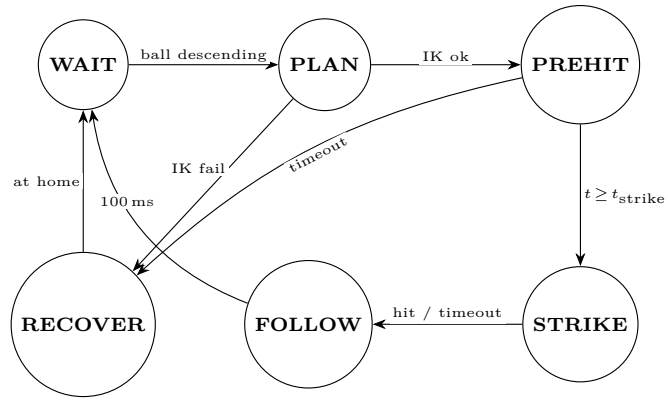


Figure 1: FSM state diagram for the baseline controller. Transitions are driven by ball tracking (descent detection, height thresholds), IK feasibility, timing schedules (strike start time derived from ballistic prediction), and contact detection (force threshold or velocity reversal).

6.8 Inverse Kinematics Solver

The IK stage within PLAN uses Drake's optimization-based `InverseKinematics` solver to convert Cartesian strike targets into feasible joint configurations for the 7-DOF KUKA iiwa arm. For each strike, the planner solves IK sequentially for three waypoints: prehit (approach pose), hit (contact pose), and follow-through (exit pose), seeding each solve from the previous solution.

The IK program enforces four constraint types:

1. **Joint limits**: bounding-box constraints on all 7 iiwa joints, tightened to ±1.40 rad around the home configuration.
2. **Seed window**: each joint is constrained within ±0.65 rad of the seed configuration, keeping solutions kinematically smooth between consecutive waypoints.
3. **Position**: the paddle frame origin must be within 12 mm of the target Cartesian point (22 mm in the relaxed fallback).
4. **Orientation**: the paddle normal must be within 1° of the desired strike normal (2° relaxed), computed from the incoming ball velocity direction with the tilt gain described above.

A quadratic cost penalizes deviation from the seed configuration (weight 2.5 per iiwa joint), biasing solutions toward postures close to the current arm state. If the solver fails to find a feasible solution for any waypoint, the FSM transitions to RECOVER rather than executing an infeasible strike.

Table 2: MuJoCo sim-to-sim transfer results under nominal MuJoCo physics (500 episodes per model, residual scale $\alpha = 0.01$).

Model	Mean Hits	Std	Survival
FSM Baseline	25.5	14.4	34%
Nominal SAC	18.6	13.9	8%
Robust SAC	24.3	12.7	29%

6.9 Why Deterministic Evaluation Is Used

Evaluation uses deterministic action selection (taking the mean of the policy's Gaussian output rather than sampling) to isolate policy quality from stochastic action sampling noise. This is particularly important when comparing checkpoints, because otherwise performance differences can be confounded by changing policy entropy realization.

However, deterministic evaluation also creates a limitation: under nominal fixed physics, some metrics become near-deterministic (e.g., near-zero standard deviation), reducing visibility into variability. We therefore include randomized evaluation and variance metrics to recover uncertainty information.

6.10 MuJoCo Sim-to-Sim Transfer

To test cross-simulator generalization, we evaluate all policies in a MuJoCo reimplementation of the ball-bouncing environment (`mujoco_transfer/`). Table 2 reports 500-episode evaluations under nominal MuJoCo physics with residual scale $\alpha = 0.01$ (Section 7). All models drop relative to Drake (Table 5), showing that sim-to-sim transfer is difficult even when nominal geometry is preserved. Under nominal MuJoCo physics, the FSM baseline achieves 25.5 mean hits with 34% survival, robust SAC reaches 24.3 mean hits with 29% survival, and nominal SAC reaches 18.6 mean hits with 8% survival.

Question. How well do Drake-trained policies transfer to a fixed nominal MuJoCo test environment? **Design.** We evaluate all models under the same MuJoCo dynamics and the same residual scale, with 500 episodes per model, to isolate transfer effects while reducing estimator variance.

Figure 2 shows the corresponding nominal MuJoCo hit comparison across models.

7 Residual Scale Analysis

The residual scale parameter α controls how much of the learned SAC correction is injected on top of the FSM+IK baseline. At $\alpha = 0$, the system is pure FSM with no RL contribution. As α increases, the RL agent applies larger joint-position corrections. This section tests how α affects MuJoCo transfer performance.

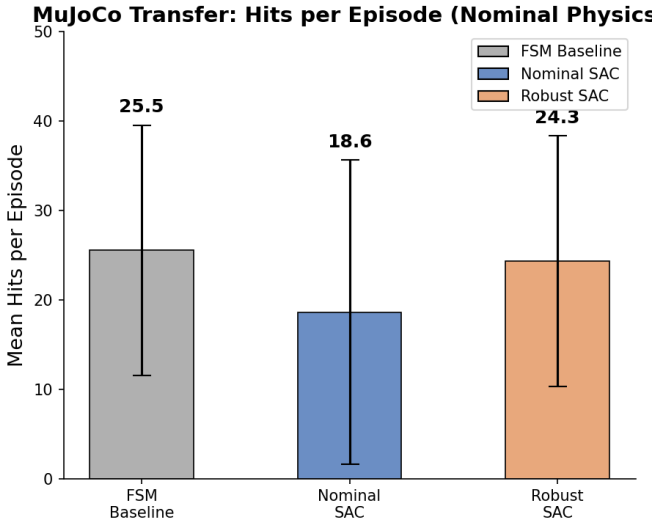


Figure 2: MuJoCo sim-to-sim transfer: hit count comparison across models under nominal physics.

Table 3: Residual scale sweep: robust SAC in MuJoCo under nominal physics (20 episodes per scale). The optimal residual scale is $\alpha = 0.01$, much smaller than the training-time value of $\alpha = 0.5$.

α	Mean Hits	Std Hits	Survival
0.000 (FSM only)	22.1	15.3	25%
0.001	23.7	14.7	30%
0.003	26.4	14.3	35%
0.005	25.3	14.0	35%
0.007	27.3	11.9	40%
0.010	**30.4**	**10.6**	**30%**
0.011	21.0	11.5	25%
0.012	23.9	11.2	25%
0.014	21.6	13.9	25%

7.1 Sweep Protocol

We swept α over 15 values from 0.0 to 0.014 using the robust SAC policy in MuJoCo with 20 episodes per scale value (Table 3). The sweep was conducted under nominal MuJoCo physics to isolate the effect of residual injection magnitude from physics perturbation.

Question. What residual-injection magnitude gives the best MuJoCo transfer performance for robust SAC? **Design.** We use a dense low-range sweep around zero, fixed nominal MuJoCo physics, and equal episode counts per scale to isolate the role of α.

7.2 Key Findings

Optimal scale is narrow. Peak performance occurs at $\alpha = 0.01$, with 30.4 mean hits, a 37.6% improvement over pure FSM ($\alpha = 0$, 22.1 hits). Performance drops outside the $[0.007, 0.010]$ band (Figure 3).

Variance reduction. Within the optimal range, stan-

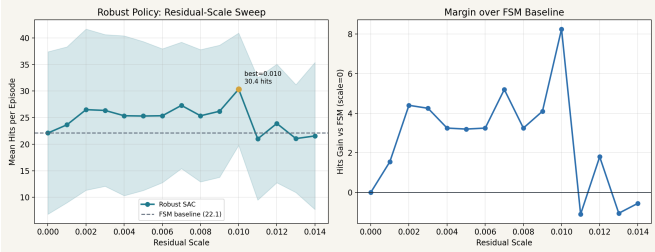


Figure 3: Residual scale sweep in MuJoCo: mean hit count (with standard deviation bands) as a function of residual scale α. Peak performance at $\alpha = 0.01$ demonstrates the sensitivity of sim-to-sim transfer to the injection magnitude.

Table 4: FSM baseline robustness (dedicated baseline report).

Condition	Mean Hits	Full 20 s Survival
Deterministic init	154 (single run)	Yes
Noisy init ($\pm$4 cm)	112.3 ± 46.7	25/50 (50%)
Stress init ($\pm$8 cm)	104.3 ± 54.9	23/50 (46%)

dard deviation drops from 15.3 (at $\alpha = 0$) to 10.6 (at $\alpha = 0.01$), suggesting more stable contact behavior.

Transfer gap explains the small optimal scale. The SAC policy was trained in Drake with $\alpha = 0.5$, but in MuJoCo the best value is $\alpha = 0.01$ (50× smaller). This is consistent with sim-to-sim mismatch: corrections tuned for Drake contacts need strong damping in MuJoCo. At larger α, RL corrections degrade baseline FSM performance.

Practical implication. For new simulators or real hardware, residual scale should be tuned at deployment time rather than copied from training. A small grid search over α worked well here.

8 Results

8.1 FSM Baseline Robustness

The baseline controller is strong and should not be treated as a trivial reference (Table 4).

Question addressed. How robust is the hand-engineered FSM+IK baseline to increasing initial-condition perturbations? **Design justification.** Reporting deterministic, noisy ($\pm$4 cm), and stress ($\pm$8 cm) start conditions provides a controlled perturbation ladder that separates nominal competence from robustness under progressively harder initialization uncertainty.

8.2 Main Quantitative Comparison

Table 5 summarizes the full evaluation matrix. We highlight four results:

Finding 1 (Absolute performance): Nominal SAC at 1M steps achieves the strongest absolute hit count in Drake across all models and conditions (Table 5), reaching 482 mean hits under nominal physics—a 3× improvement over the FSM baseline (159 hits).

Finding 2 (Relative robustness): Robust SAC becomes positive in Δ at later checkpoints: +25.9% at 750k and +48.5% at 1M (Figure 4b), meaning it performs *better* under randomized physics than nominal.

Finding 3 (Learning dynamics): Both tracks exhibit a 750k dip and 1M rebound (Figure 4a), consistent with the value-function reorganization discussed in Section 3.5.

Finding 4 (Consistency): Nominal SAC 1M has high randomized variance ($\sigma = 106$); robust SAC 1M is lower ($\sigma = 30.9$), as shown in Figure 4c. This is a practical benefit of domain randomization.

8.3 Extended Metric Analysis

To separate rally quantity from episode duration, we compute mean hit throughput $\rho = H/T$, where H is the mean hit count and T is the mean simulation time in seconds (Table 6).

Table 6 clarifies a subtle behavior: robust policies can exhibit short episodes with very high instantaneous hit throughput (see also Figure 4d). Thus, high hit count and long survival are related but not identical objectives in this setup.

8.4 Checkpoint-by-Checkpoint Interpretation

500k stage. Nominal 500k is already competitive with baseline in throughput, while robust 500k remains conservative in absolute hit count. This matches the usual early trade-off in robust training.

750k stage. Both nominal and robust trajectories dip in nominal evaluation. Robust 750k is notable because randomized performance exceeds nominal performance (+25.9%), suggesting adaptation to the randomization distribution even before absolute performance matures.

1M stage. Nominal 1M undergoes a clear breakthrough in absolute hits under both conditions but retains high OOD variance ($\sigma = 106$). Robust 1M improves both mean hits and relative perturbation behavior but still trails nominal 1M in absolute peak.

8.5 Cross-Metric Tensions

The results (Figure 4) expose three objective tensions:

- **Peak hits vs perturbation robustness:** nominal 1M dominates one, robust 1M trends better on the other.
- **Throughput vs episode duration:** some policies hit rapidly in shorter windows.
- **Mean vs predictability:** higher mean can coexist with larger variance.

Table 5: 20-episode evaluation summary across models and physics conditions in the Drake simulator.

Model	Eval Physics	Mean Hits	Std Hits	Max Hits	Mean Sim Time (s)	Δ vs Nominal
FSM Baseline	Nominal	159.0	0.0	159	20.0	–
FSM Baseline	Randomized	145.3	29.0	189	17.7	−8.6%
Nominal SAC 500k	Nominal	144.0	0.0	144	11.0	–
Nominal SAC 500k	Randomized	112.8	55.1	274	8.5	−21.7%
Nominal SAC 750k	Nominal	110.0	0.0	110	6.6	–
Nominal SAC 750k	Randomized	61.3	35.9	156	3.9	−44.3%
Nominal SAC 1M	Nominal	**482.0**	0.0	**482**	20.0	–
Nominal SAC 1M	Randomized	**328.0**	**106.0**	**474**	13.9	−32.0%
Robust SAC 500k	Nominal	53.0	0.0	53	3.5	–
Robust SAC 500k	Randomized	43.4	11.0	66	2.8	−18.2%
Robust SAC 750k	Nominal	23.0	0.0	23	1.7	–
Robust SAC 750k	Randomized	29.0	7.5	51	1.8	+25.9%
Robust SAC 1M	Nominal	76.0	0.0	76	1.7	–
Robust SAC 1M	Randomized	112.9	30.9	181	2.7	+48.5%

Table 6: Derived hit throughput (hits/s) from evaluation means.

Model	Nominal ρ	Randomized ρ
FSM Baseline	7.95	8.23
Nominal 500k	13.10	13.28
Nominal 750k	16.57	15.80
Nominal 1M	24.10	23.60
Robust 500k	15.19	15.25
Robust 750k	13.37	16.05
Robust 1M	45.24	41.50

9 Failure Analysis and Lessons

The original goal was a single policy that is best in absolute performance and robustness. Under the present budget, this did not happen. Two failure modes emerged:

Failure mode A: Nominal over-specialization. Nominal SAC learns high-return trajectories but degrades under perturbation (−32.0% at 1M). This is consistent with specialization to fixed contact dynamics.

Failure mode B: Robust under-optimization. Robust SAC improves relative robustness but remains below nominal SAC in peak absolute hits at 1M.

Interventions attempted. We increased the training budget to 1M and retained intermediate checkpoint evaluation; used wide randomization ranges over contact and inertial parameters; and compared both training types under both evaluation conditions.

Interpretation. Domain-randomized training appears to spend capacity on cross-physics invariance, which can delay convergence to highly specialized nominal strategies.

10 Threats to Validity

- **Single-seed training**: uncertainty across random initializations is not fully characterized.
- **Deterministic nominal evaluation**: near-zero nominal std is expected but limits uncertainty reporting.
- **Simulation-only conclusions**: sim-to-real transfer is not yet validated on hardware.
- **Metric coupling**: hit counting and horizon settings influence rankings.
- **Residual scale sensitivity**: the optimal deployment scale ($\alpha = 0.01$ in MuJoCo vs $\alpha = 0.5$ in training) highlights that transfer requires careful recalibration.

11 Model-Family Case Analysis

11.1 Nominal SAC Family

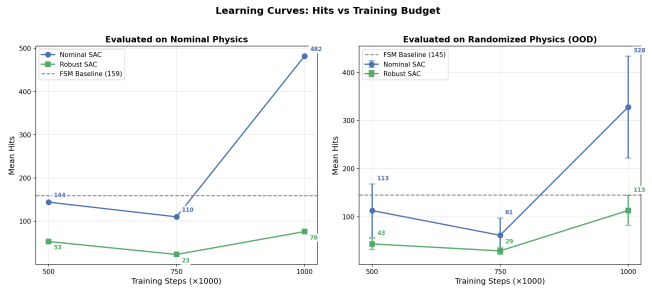
The nominal SAC checkpoints illustrate a specialization trajectory. At 500k and 750k, the policy has moderate hit throughput and strong degradation under perturbation. At 1M, the policy reaches a qualitatively different regime with very high hit counts in both nominal and randomized tests, but with substantial variance under randomized physics ($\sigma = 106$).

This pattern suggests the learned strategy eventually discovers a high-frequency control mode that strongly exploits nominal simulator structure. The same strategy can still perform well on average under perturbation, but instability increases, consistent with brittle contact sensitivity.

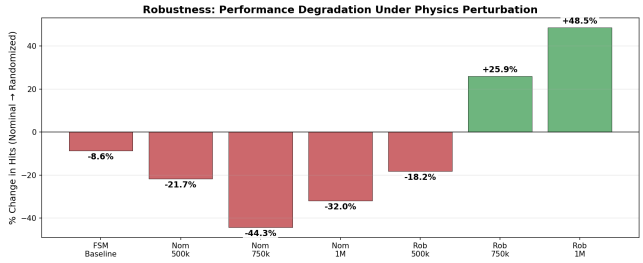
11.2 Robust SAC Family

The robust SAC family behaves differently. Early checkpoints are conservative and low in absolute hits, then later checkpoints improve relative perturbation behavior (positive Δ). This can be interpreted as the policy gradually internalizing dynamics-invariant corrective behavior.

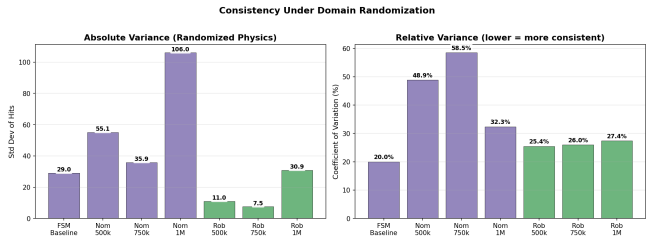
Although robust 1M underperforms nominal 1M on absolute hit count, it is less volatile under randomized



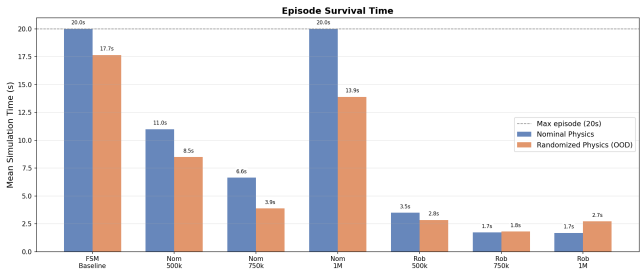
(a) Learning curves versus training budget.



(b) Relative perturbation effect Δ.



(c) Absolute and relative variability under randomized physics.



(d) Episode duration comparison.

Figure 4: Core experimental results: performance curves, robustness degradation, variance profiles, and episode survival times across all models and evaluation conditions.

physics ($\sigma = 30.9$ vs $\sigma = 106$). For applications prioritizing reliability under model mismatch, this may be preferable despite lower peak returns.

11.3 Baseline vs Learned Controllers

The FSM+IK baseline remains a useful anchor because it is:

- interpretable (state-machine phases and IK objectives are inspectable),
- stable in nominal deployment,
- reasonably robust under moderate perturbation (-8.6% degradation, Table 5).

Residual learning should therefore be judged not only by raw score uplift, but by whether it preserves or improves these baseline reliability properties.

11.4 Practical Selection Guidance

Given current evidence, a deployment-oriented selection rule could be:

- choose **Nominal 1M** when maximizing expected throughput in environments close to nominal simulator dynamics;
- choose **Robust 1M** when dynamics uncertainty is large and performance consistency is prioritized;
- retain **FSM+IK** as a fallback safety baseline.

This policy-selection framing is often more actionable than a single "best model" declaration.

12 Reproducibility and Experimental Governance

12.1 Data Provenance

The quantitative values in this paper are taken from repository artifacts:

- `results/all_eval_results.json`
- `results/fsm_baseline_report.md`
- `results/rl_training_report.md`
- generated figures in `results/final_*.png`
- residual scale tuning data in `results/residual_scale_tuning/`
- MuJoCo 500-episode evaluations in `results/mujoco_eval_protocol_*/`

This provenance path makes each table and plot auditable.

12.2 Evaluation Integrity Checks

At minimum, evaluation should check:

- deterministic-evaluation flag consistency across runs,
- checkpoint naming and step-count alignment,
- fixed episode count across compared rows,
- identical metric definitions (especially hit event detection),
- correct mapping between plot labels and table rows.

These checks are straightforward but essential for avoiding silent comparison errors.

12.3 Recommended Reporting Standard

For stronger final claims, future iterations should report:

- multi-seed mean and confidence intervals,
- per-episode distributions (not only mean/std aggregates),
- sensitivity to reward weights and residual envelope,
- additional robustness stress tests beyond current randomization ranges.

13 Future Directions

- multi-seed confidence intervals for all checkpoints/conditions;
- curriculum randomization schedules (narrow-to-wide);
- risk-sensitive objectives targeting lower-tail performance;
- systematic residual scale adaptation (e.g., learned or scheduled α);
- MuJoCo transfer plus real-robot calibration pipeline.

14 Conclusion

This project studies a hybrid controller (FSM+IK baseline plus SAC residual). In Drake, residual learning improved over the baseline, with nominal SAC at 1M reaching 482 mean hits, about 3× the FSM baseline (159).

Domain randomization helps. The robust SAC policy, trained with per-episode physics randomization, reaches positive perturbation effects (+25.9% at 750k and +48.5% at 1M). In other words, it performs better under randomized evaluation than under nominal evaluation at later checkpoints.

Consistency advantage. Under randomized physics, robust SAC 1M has much lower variance than nominal SAC 1M ($\sigma = 30.9$ vs $\sigma = 106$), about 3.5× lower.

Residual scale in transfer. The residual scale analysis (Section 7) shows that RL injection must be recalibrated after transfer. In MuJoCo, the best scale is $\alpha = 0.01$, which is 50× smaller than the training value ($\alpha = 0.5$). At this setting, the residual policy improves over FSM by 37.6%.

Future reward shaping for apex quality. A next step is to increase the apex-height reward weight. Right now, the policy can gain reward from many short low-height taps (few-centimeter hits) within 20 seconds instead of fewer higher bounces that better match the target apex.

Overall, the main result is a clear trade-off: nominal training gives higher peak hits, while randomized training gives better robustness and lower variance.

15 Project Retrospective

Biggest challenge. The biggest challenge was understanding why MuJoCo transfer underperformed. At first we thought simulator mismatch was the only issue, but debugging showed a second issue: the apex-height reward weight was too small. The policy then preferred many small contacts instead of higher-quality bounces, and that behavior did not pair well with PID tuning built around the FSM's larger-hit pattern. Learning MuJoCo itself was also a challenge because none of us had used it before.

Most pleased with. We are most pleased with the hybrid controller result in Drake. Adding a residual RL policy on top of IK/FSM improved performance while keeping the structure and reliability of the baseline.

What we would do differently. Next time we would do stricter diagnostics in Drake before transfer. In particular, we would test reward sensitivity (especially apex-height weighting), inspect hit-amplitude distributions, and check controller-policy interaction earlier. That would likely have exposed the reward-design issue before the MuJoCo debugging phase.

Code Availability and Reproducibility

All code, trained models, and evaluation artifacts are publicly available at https://github.com/MG-05/ping-pong-domain-randomization. To reproduce the full experimental pipeline (training, evaluation, and report generation), run the following commands with the repository dependencies installed:

```
python -m src.train_rl --steps 1000000 \
  --no-randomize \
  --save-path data/sac_nominal_1m --seed 42

python -m src.train_rl --steps 1000000 \
  --randomize \
  --save-path data/sac_robust_1m --seed 42

python scripts/run_all_evals.py
python results/generate_final_report.py
```

A Proofs and Notation

Table 7 summarizes the key notation used throughout this paper.

A.1 Proof of Residual Boundedness (Proposition 1)

Proposition 2 (Residual boundedness)**.** *If $q_t^{\text{base}} \in \mathcal{Q}$, then $q_t^{\text{cmd}} \in \mathcal{Q}$ for all t and for any policy output a_t.*

Proof. Let $q_t^{\text{base}} \in \mathcal{Q}$ be the baseline joint command produced by the FSM+IK controller, where $\mathcal{Q} = \{q \in \mathbb{R}^7 : q_i^{\min} \le q_i \le q_i^{\max},\ i = 1, \dots, 7\}$.

The residual action $a_t \in \mathbb{R}^7$ is first clipped: $\bar{a}_t = \text{clip}(a_t, -1, 1)$, so $\bar{a}_t^{(i)} \in [-1, 1]$ for each joint i.

Table 7: Summary of notation.

Symbol	Meaning
$\mathcal{M}$	MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$
$s_t \in \mathbb{R}^{20}$	State: joint pos/vel (14) + ball pos/vel (6)
$a_t \in [-1, 1]^7$	Normalized residual action (7 joints)
q_t^{base}	FSM+IK baseline joint command ($\mathbb{R}^7$)
q_t^{cmd}	Final joint command after residual ($\mathbb{R}^7$)
α	Residual scale $\in [0, 1]$
δ	Maximum residual magnitude (0.15 rad)
$\Pi_{\mathcal{Q}}$	Projection onto joint-limit set $\mathcal{Q}$
$\mathcal{Q}$	Feasible joint-limit set $\{q : q^{\min} \leq q \leq q^{\max}\}$
γ	Discount factor (0.99)
β	SAC entropy temperature (auto-tuned)
τ	Polyak averaging rate (0.005)
Q^π, V^π	Soft Q-function and soft value function
$\mathcal{D}$	Replay buffer
ϕ, θ	Critic and actor network parameters
$\bar{\phi}_i$	Target critic network parameters
$P_{\text{nom}}, P_{\text{rand}}$	Nominal / randomized transition kernels
ξ	Physics parameter vector
$\mathcal{P}_\Xi$	Distribution over physics parameters
$R(\tau)$	Discounted cumulative return along trajectory τ
H	Mean hit count over N episodes
$\Delta(\%)$	Relative perturbation effect
CV(%)	Coefficient of variation under randomized physics
ρ	Hit throughput (hits/s)
$q_{1:7}, \dot{q}_{1:7}$	KUKA iiwa joint positions and velocities
p_b, v_b	Ball position and velocity in world frame
z_{hit}	Target hit height (0.38 m)
τ_{hit}	Time-to-intercept (s)
g	Gravitational acceleration (9.81 m/s^2)

The unclamped command is:

$$\tilde{q}_t = q_t^{\text{base}} + \alpha \cdot \delta \cdot \bar{a}_t,$$

where $\alpha \in [0, 1]$ and $\delta = 0.15$ rad. Each component satisfies:

$$|\tilde{q}_t^{(i)} - q_t^{\text{base},(i)}| = \alpha \cdot \delta \cdot |\bar{a}_t^{(i)}| \leq \alpha \cdot \delta \leq 0.15 \text{ rad}.$$

Thus, the residual perturbation is bounded per joint.

The final command is computed by projection:

$$q_t^{\text{cmd}} = \Pi_{\mathcal{Q}}(\tilde{q}_t),$$

where $\Pi_{\mathcal{Q}}$ clamps each joint independently:

$$q_t^{\text{cmd},(i)} = \max\big(q_i^{\min},\ \min(q_i^{\max},\ \tilde{q}_t^{(i)})\big).$$

By construction, $q_t^{\text{cmd},(i)} \in [q_i^{\min}, q_i^{\max}]$ for all i. Therefore $q_t^{\text{cmd}} \in \mathcal{Q}$. □

A.2 Proof of Expected-Risk Interpretation (Proposition 1)

Proof. Let $\xi_1, \xi_2, \ldots, \xi_K$ be K i.i.d. samples from $\mathcal{P}_\Xi$, where each ξ_k specifies the physics parameters for training episode k.

The expected return of policy π under the physics distribution is:

$$J_{\text{robust}}(\pi) = \mathbb{E}_{\xi \sim \mathcal{P}_\Xi}\left[\mathbb{E}_{\tau \sim P_\xi, \pi}[R(\tau)]\right].$$

During domain-randomized training, at each episode k the environment samples $\xi_k \sim \mathcal{P}_\Xi$ and runs the policy to collect trajectory τ_k under transition kernel P_{ξ_k}. The empirical return is $R(\tau_k)$.

The Monte Carlo estimate of $J_{\text{robust}}(\pi)$ is:

$$\hat{J}_K(\pi) = \frac{1}{K} \sum_{k=1}^{K} R(\tau_k).$$

Since each ξ_k is i.i.d. from $\mathcal{P}_\Xi$ and each τ_k is sampled from P_{ξ_k} under π:

$$\mathbb{E}[\hat{J}_K(\pi)] = \frac{1}{K} \sum_{k=1}^{K} \mathbb{E}_{\xi_k \sim \mathcal{P}_\Xi}\left[\mathbb{E}_{\tau_k \sim P_{\xi_k}, \pi}[R(\tau_k)]\right] = J_{\text{robust}}(\pi).$$

Therefore, $\hat{J}_K(\pi)$ is an unbiased estimator of $J_{\text{robust}}(\pi)$. By the law of large numbers, $\hat{J}_K(\pi) \xrightarrow{K \to \infty} J_{\text{robust}}(\pi)$ almost surely.

In practice, the policy π_θ is parameterized by neural network weights θ and optimized via SAC gradient updates using individual episode returns. Function-approximation error (from the critic and actor networks) and finite training episodes introduce bias relative to the true optimal π^*_{robust}, but the training objective itself targets J_{robust}. □

References

[1] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *Proceedings of ICML*, 2018.

[2] Timothy Lillicrap, Jonathan Hunt, Alexander Pritzel, et al. Continuous Control with Deep Reinforcement Learning. arXiv:1509.02971, 2015.

[3] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World. In *Proceedings of IROS*, 2017.

[4] Xue Bin Peng, Marcin Andrychowicz, Wojciech Zaremba, and Pieter Abbeel. Sim-to-Real Transfer of Robotic Control with Dynamics Randomization. In *Proceedings of ICRA*, 2018.

[5] Lerrel Pinto, James Davidson, Rahul Sukthankar, and Abhinav Gupta. Robust Adversarial Reinforcement Learning. In *Proceedings of ICML*, 2017.

[6] Tobias Johannink, Shikhar Bahl, Ashvin Nair, et al. Residual Reinforcement Learning for Robot Control. In *Proceedings of ICRA*, 2019.

[7] Iman Nematollahi, Erick Rosete-Beas, Adrian Roefer, Tim Welschehold, Abhinav Valada, and Wolfram Burgard. Robot Skill Adaptation via Soft Actor-Critic Gaussian Mixture Models. In *Proceedings of ICRA*, pages 8651–8657, 2022. https://doi.org/10.1109/ICRA46639.2022.9811770.

[8] Todor Davchev, Kevin Sebastian Luck, Michael Burke, Franziska Meier, Stefan Schaal, and Subramanian Ramamoorthy. Residual Learning from Demonstration: Adapting DMPs for Contact-rich Manipulation. *IEEE Robotics and Automation Letters*, 7(2):4488–4495, 2022. https://doi.org/10.1109/LRA.2022.3150024.

[9] Mohammadreza Nakhaei, Aidan Scannell, and Joni Pajarinen. Residual Learning and Context Encoding for Adaptive Offline-to-Online Reinforcement Learning. In *Proceedings of the 6th Annual Learning for Dynamics and Control Conference (L4DC)*, volume 242 of *Proceedings of Machine Learning Research*, pages 1107–1121, 2024. https://proceedings.mlr.press/v242/nakhaeinezhadfard24a.html.

[10] Antonin Raffin, Ashley Hill, Adam Gleave, et al. Stable-Baselines3: Reliable Reinforcement Learning Implementations. https://github.com/DLR-RM/stable-baselines3, 2021.

[11] Richard Sutton and Andrew Barto. Reinforcement Learning: An Introduction. MIT Press, 2nd edition, 2018.